

Student Record Manager

Python Mini Assignment Documentation

Assignment type Console application	Language Python 3	Storage JSON file
---	-----------------------------	-----------------------------

Purpose

This assignment demonstrates how a small Python program can organize student records through a text menu. A user can create, view, search, edit, and remove records. Data is saved in a JSON file next to the program so it is available in later sessions.

What you will learn

Functions and program structure; lists and dictionaries; loops and conditional logic; input validation; searching; and reading and writing JSON data.

Designed for introductory Python practice

Requirements and design

Functional requirements

Action	Description
Add	Collect name, course, and marks; assign the next available numeric ID.
List	Show saved records in a readable table.
Search	Match a text term against student names and course names without case sensitivity.
Update	Edit a record by ID; blank input keeps the existing value.
Delete	Remove a record by ID.
Exit	End the menu loop cleanly.

Data structure

Each student is stored as a dictionary with four fields: id (integer), name (text), course (text), and marks (number). The application keeps a list of these dictionaries in memory and serializes the list as indented JSON in `students.json` after each change.

Validation and error handling

Names and course names cannot be blank when adding a student. Marks must be numeric and between 0 and 100. Menu choices and record IDs are checked before use. If saved JSON cannot be read, the application warns the user and starts with an empty list.

Files

`student_manager.py` contains the application. **`students.json`** is generated at runtime and holds records. **`README.md`** contains quick-start instructions and a sample-run image.

How to run and use the program

Open a terminal in the assignment folder and run:

```
python student_manager.py
```

Menu option	Result
1	Add a student. Enter name, course, and marks from 0 to 100.
2	List all students.
3	Search names and courses by a text fragment.
4	Update a student using their ID. Press Enter to retain a field.
5	Delete a student using their ID.
0	Exit the program.

Example record

```
{
  "id": 1,
  "name": "Asha Patel",
  "course": "Computer Science",
  "marks": 92.0
}
```

Possible extensions

Add sorting by marks, calculate class averages, export a CSV report, or introduce confirmation before deletion. These are optional improvements beyond the core assignment.